HW4: Palmer Penguins — Summary Statistics & Plots

Setup

- Use only `pandas`, `matplotlib`, and `seaborn`.
- Do your work in the cells provided; you may add extra cells.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# (Optional) nicer display for DataFrames
pd.set_option("display.max_columns", 50)
pd.set_option("display.precision", 3)
```

In [2]:
```
### Load the dataset
penguins = pd.read_csv("penguins.csv")

# Standardize column names (snake_case)
penguins.columns = [c.strip().lower().replace(' ', '_') for c in penguins.columns]

penguins.head()
```

Out[2]:

	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	sex	year
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	male	2007
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	female	2007
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	female	2007
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	NaN	2007
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	female	2007

Part 1) Data Vitals

Answer the following questions.

Your answer to each question must include some code that produces the result. You can, but you do not have to use a complete sentence to answer each question. Add cells as needed.

- a. What is the shape of the data?
- b. What are the Column names and data types of each column?
- c. How many values in each column are missing?
- d. How many penguins are there of each species? How many male and female penguins are there?

a)

In [3]:
```
penguins.shape
```

Out[3]:
```
(344, 8)
```

There are 344 rows and 8 columns.

b)

In [4]:
```
penguins.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 344 entries, 0 to 343
Data columns (total 8 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   species            344 non-null    str
 1   island             344 non-null    str
 2   bill_length_mm     342 non-null    float64
 3   bill_depth_mm      342 non-null    float64
 4   flipper_length_mm  342 non-null    float64
 5   body_mass_g        342 non-null    float64
 6   sex                333 non-null    str
 7   year               344 non-null    int64
dtypes: float64(4), int64(1), str(3)
memory usage: 21.6 KB
```

c)

In [5]:
```
penguins.isna().sum(axis = 0)
```

Out[5]:
```
species              0
island               0
bill_length_mm       2
bill_depth_mm        2
flipper_length_mm    2
body_mass_g          2
sex                 11
year                 0
dtype: int64
```

d)

In [6]:
```
penguins['species'].value_counts()
```

Out[6]:
```
species
Adelie       152
Gentoo       124
Chinstrap     68
Name: count, dtype: int64
```

In [7]:
```
penguins['sex'].value_counts()
```

Out[7]:
```
sex
male      168
female    165
Name: count, dtype: int64
```

Part 2) Handle missing data

- a. Create a copy `penguins_clean` that **drops** rows with missing values in `['bill_length_mm','bill_depth_mm','flipper_length_mm','body_mass_g','sex']`.
- b. Report the new shape of `penguins_clean`.

a)

In [8]:
```
penguins_clean = penguins.dropna()
```

b)

In [9]:
```
penguins_clean.shape
```

Out[9]:
```
(333, 8)
```

Part 3) Numeric summary statistics

- a. Compute the following **for each numeric column**: count, mean, std, min, 25%, 50%, 75%, max (hint: `.describe()`)
- b. Compute the mean and std for each numeric column for each `species`

a)

In [10]:
```
penguins_clean.describe()
```

Out[10]:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
count	333.000	333.000	333.000	333.000	333.000
mean	43.993	17.165	200.967	4207.057	2008.042
std	5.469	1.969	14.016	805.216	0.813
min	32.100	13.100	172.000	2700.000	2007.000
25%	39.500	15.600	190.000	3550.000	2007.000
50%	44.500	17.300	197.000	4050.000	2008.000
75%	48.600	18.700	213.000	4775.000	2009.000
max	59.600	21.500	231.000	6300.000	2009.000

b)

In [11]:
```
penguins_clean.groupby(penguins_clean['species']).mean(numeric_only = True)
```

Out[11]:

species	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
Adelie	38.824	18.347	190.103	3706.164	2008.055
Chinstrap	48.834	18.421	195.824	3733.088	2007.971
Gentoo	47.568	14.997	217.235	5092.437	2008.067

In [12]:
```
penguins_clean.groupby(penguins_clean['species']).std(numeric_only = True)
```

Out[12]:

species	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
Adelie	2.663	1.219	6.522	458.620	0.812
Chinstrap	3.339	1.135	7.132	384.335	0.863
Gentoo	3.106	0.986	6.585	501.476	0.789

Part 4) Two-way grouped summaries

a) For each **(species, sex)** combination, compute the **mean** and **count** of:

- `bill_length_mm`, `bill_depth_mm`, `flipper_length_mm`, `body_mass_g`.

b) Which (species, sex) has the **largest average body_mass_g**? Show the row.

a)

In [13]:
```
penguins_clean.groupby(by = ['species', 'sex']).mean(numeric_only = True).drop(columns = "year")
```

Out[13]:

species	sex	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
Adelie	**female**	37.258	17.622	187.795	3368.836
	male	40.390	19.073	192.411	4043.493
Chinstrap	**female**	46.574	17.588	191.735	3527.206
	male	51.094	19.253	199.912	3938.971
Gentoo	**female**	45.564	14.238	212.707	4679.741
	male	49.474	15.718	221.541	5484.836

In [14]:
```
penguins_clean.groupby(by = ['species', 'sex']).count().drop(columns = ["island","year"])
```

Out[14]:

species	sex	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g
Adelie	**female**	73	73	73	73
	male	73	73	73	73
Chinstrap	**female**	34	34	34	34
	male	34	34	34	34
Gentoo	**female**	58	58	58	58
	male	61	61	61	61

b)

In [15]:
```
mean_table = penguins_clean.groupby(by = ['species', 'sex']).mean(numeric_only = True).drop(columns = "year"
index = mean_table.idxmax(axis = 0)['body_mass_g']
mean_table.loc[index]
```

Out[15]:
```
bill_length_mm         49.474
bill_depth_mm          15.718
flipper_length_mm     221.541
body_mass_g          5484.836
Name: (Gentoo, male), dtype: float64
```

Part 5) Correlations

- a. Compute the **correlation matrix** among numeric columns.
- b. Compute the correlation matrix **within each species** (hint: groupby).
- c. Find an example of Simpson's Paradox in the data. https://en.wikipedia.org/wiki/Simpson%27s_paradox
  - An example Simpson's Paradox would be a pair of variables that have a positive correlation within subgroups, but when all the groups are combined, there is negative correlation (or vice-versa).

a)

In [16]:
```
penguins_clean.corr(numeric_only = True)
```

Out[16]:

	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
bill_length_mm	1.000	-0.229	0.653	0.589	0.033
bill_depth_mm	-0.229	1.000	-0.578	-0.472	-0.048
flipper_length_mm	0.653	-0.578	1.000	0.873	0.151
body_mass_g	0.589	-0.472	0.873	1.000	0.022
year	0.033	-0.048	0.151	0.022	1.000

b)

In [17]:
```
penguins_clean.groupby(by = "species").corr(numeric_only = True)
```

Out[17]:

species		bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	year
Adelie	**bill_length_mm**	1.000	0.386	0.332	0.544	0.011
	bill_depth_mm	0.386	1.000	0.311	0.580	-0.235
	flipper_length_mm	0.332	0.311	1.000	0.465	0.328
	body_mass_g	0.544	0.580	0.465	1.000	-0.046
	year	0.011	-0.235	0.328	-0.046	1.000
Chinstrap	**bill_length_mm**	1.000	0.654	0.472	0.514	0.042
	bill_depth_mm	0.654	1.000	0.580	0.604	-0.059
	flipper_length_mm	0.472	0.580	1.000	0.642	0.346
	body_mass_g	0.514	0.604	0.642	1.000	0.037
	year	0.042	-0.059	0.346	0.037	1.000
Gentoo	**bill_length_mm**	1.000	0.654	0.664	0.667	0.205
	bill_depth_mm	0.654	1.000	0.711	0.723	0.243
	flipper_length_mm	0.664	0.711	1.000	0.711	0.201
	body_mass_g	0.667	0.723	0.711	1.000	0.052
	year	0.205	0.243	0.201	0.052	1.000

c)

An example of Simpson's Paradox in this data is the correlation between bill length and bill depth. In each species, the correlation between the two is positive, with the correlation being 0.386, 0.654, and 0.654 in the Adelie, Chinstrap, and Gentoo species respectively. However, overall the correlatoin between bill length and bill depth is -0.229.

In [ ]:
```

```

In [ ]:
```

```

Plots

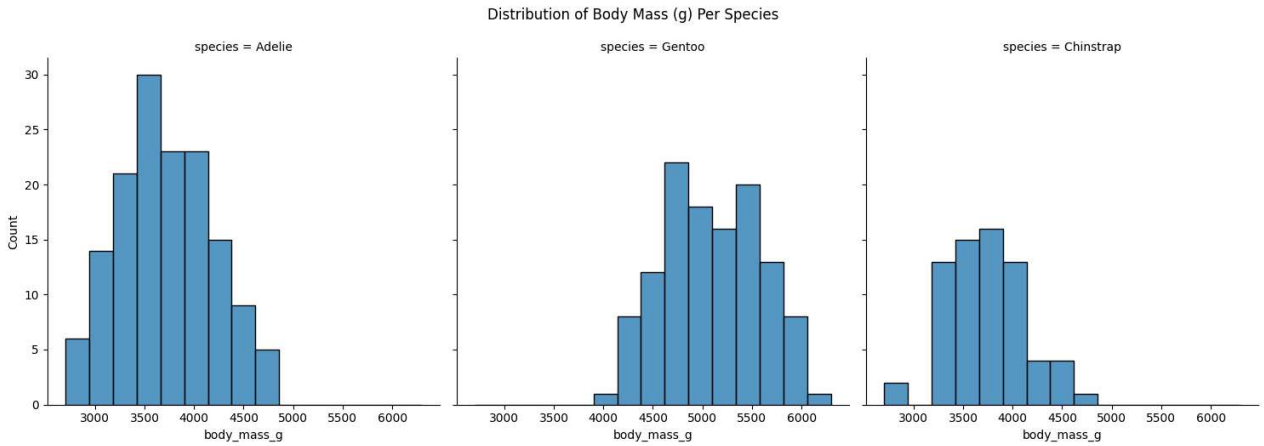
- Each figure must have a **title**, **axis labels**, and a **legend** (when appropriate).
- Use readable tick labels and sensible limits.
- Be sure each graphic is displayed.

Part 6) Histograms of body mass by species

Create **separate** histograms of `body_mass_g` for each species (three panels).

- Use the same binning across panels so comparisons are fair.
- Label axes clearly

In [18]:
```
p = sns.displot(data = penguins_clean, x = "body_mass_g", bins = 15, col = "species")
p.figure.suptitle("Distribution of Body Mass (g) Per Species", y = 1.05)
plt.show()
```

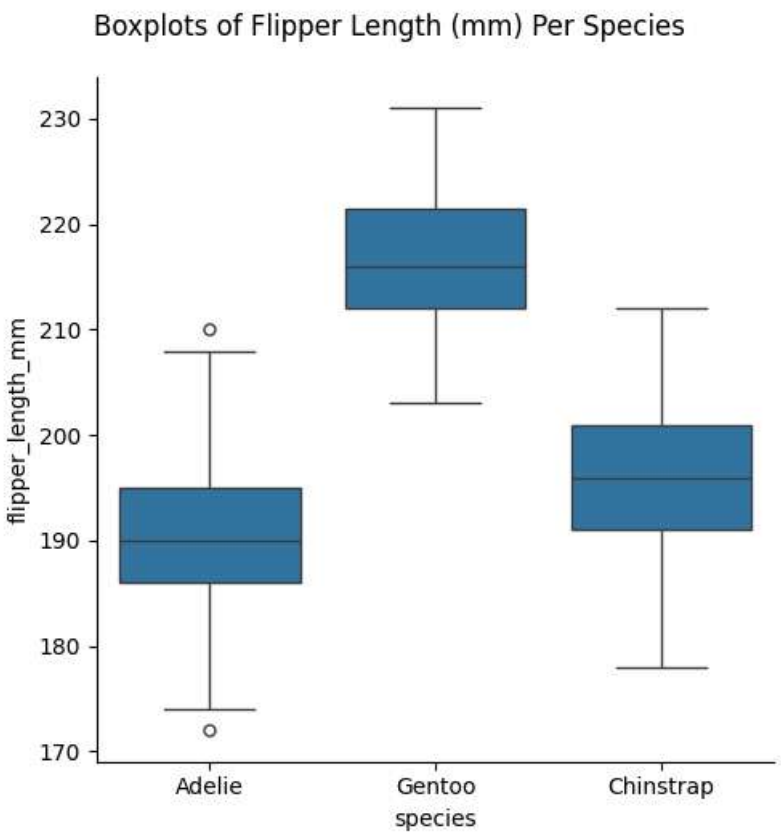


Part 7) Boxplots of flipper length

Make a **single figure** with boxplots of `flipper_length_mm` by **species.**

Briefly compare the medians and spreads in 2–3 sentences in a markdown cell.

In [19]:
```
p = sns.catplot(data = penguins_clean, x = "species", y = "flipper_length_mm", kind = "box")
p.figure.suptitle("Boxplots of Flipper Length (mm) Per Species", y = 1.05)
plt.show()
```

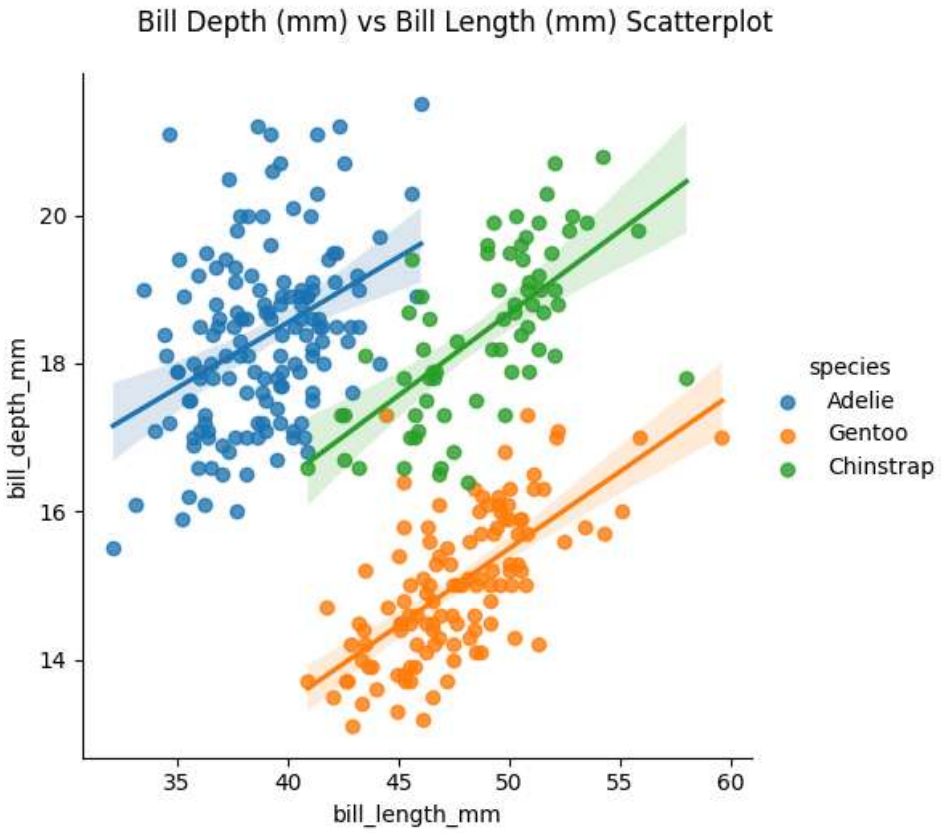


The median of Adelie and Chinstrap are similar while the median for Gentoo is bigger. The spread of Adelie and Gentoo seems to be a bit right skewed while Chinstrap might be more normal. Adelie is the only species to have outliers.

Part 8) Scatter: bill length vs bill depth

- Make a scatter plot of **bill_length_mm** (x) vs **bill_depth_mm** (y), colored by species.
- Add a simple **linear fit line** per species
- Include a legend

In [20]:
```
p = sns.lmplot(data = penguins_clean, x = "bill_length_mm", y = "bill_depth_mm", hue = "species")
p.figure.suptitle("Bill Depth (mm) vs Bill Length (mm) Scatterplot", y = 1.05)
plt.show()
```

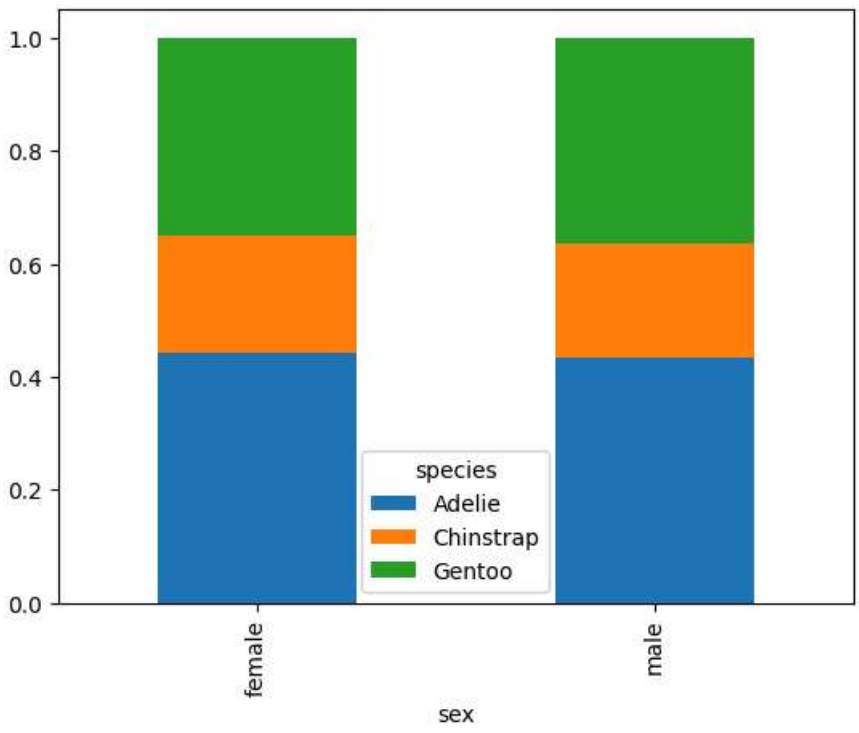


Part 9) Stacked bars: sex proportions by species

Compute counts of `sex` within each `species`, convert to **proportions**, and make a **stacked bar chart.**

- Bars should be species on the x-axis with male/female proportions stacked to 1.0.

In [21]:
```
p = pd.crosstab(penguins_clean.sex, penguins_clean.species, normalize = 0).plot(kind = "bar", stacked = True
p.figure.suptitle("Proportion of Sex Within Each Species", y = 1.05)
plt.show()
```


Proportion of Sex Within Each Species
1.0
0.8
0.6
0.4
0.2
0.0
species
Adelie
Chinstrap
Gentoo
female
male
sex